UNIVERSIDAD INTERAMERICANA DE PUERTO RICO · RECINTO DE FAJARDO

PROYECTO FINAL · DESARROLLO WEB FULL-STACK

Sabías que PR

Evidencia de pruebas

Fase 4: Pruebas y publicación · pruebas funcionales, corrección de errores y publicación en línea

Estudiante	William R. Collado
Curso	COMP2053 — Desarrollo Web Full-Stack
Profesora	Milagros Donato Cintrón
Institución	Universidad Interamericana de Puerto Rico, Recinto de Fajardo
Repositorio	github.com/Wcollado72/SabiasquePR-Final
Sitio en línea	sabiasquepr.com
Fecha	24 de octubre de 2026

1. Resumen

Este documento reúne la evidencia de las pruebas de **Sabías que PR**: las pruebas automáticas del backend (Jest y Supertest), la verificación del sitio y del API publicados desde fuera, y las pruebas de carga y de pantalla en navegadores. Todas las cifras se **leen de los resultados guardados** por los scripts del repositorio (ver la sección 7); nada se escribió a mano. Lo que no se pudo probar se dice con claridad en la sección 5.

85/85	1269/1269	93,0 %	21 s
suites de pruebas pasan	pruebas pasan (0 fallan, 0 omitidas)	cobertura de líneas (2248 de 2418)	duración de la ejecución completa

Ejecución del 24/10/2026 15:50 con Node v24.13.0. Comando: `cd backend && npx jest --coverage`.

Cobertura de código

Cobertura global: **93,0 %** de líneas, 91,8 % de sentencias, 92,6 % de funciones y 86,3 % de ramas, medida sobre `backend/src/**` excepto `server.js`. Por carpeta:

Carpeta	Archivos	Líneas	Sentencias	Funciones	Ramas
`src`	2	96,6 %	96,7 %	77,8 %	92,9 %
`src/lib`	1	100,0 %	100,0 %	100,0 %	100,0 %
`src/middleware`	1	94,7 %	87,7 %	100,0 %	84,0 %
`src/routes`	17	93,6 %	92,6 %	95,0 %	88,7 %
`src/services`	18	87,0 %	85,8 %	88,5 %	75,9 %
`src/test-utils`	1	89,5 %	90,9 %	100,0 %	100,0 %
`src/util`	9	97,7 %	94,3 %	100,0 %	86,7 %
`src/validators`	17	99,8 %	99,0 %	100,0 %	97,3 %

Las carpetas de rutas y validadores, donde viven el CRUD y la validación, superan el 90 % de líneas. La carpeta más baja es `src/services` (87 % de líneas): son los agentes de IA (Investigador, Redactor, Traductor) y el extractor de artículos, que dependen de la API de Claude y de páginas externas; sus pruebas cubren la lógica interna con servicios simulados, no las llamadas reales.

Qué significan y qué no significan estas pruebas

Las pruebas **no necesitan red ni base de datos real**: ninguna se conectó a Supabase, a Stripe ni a la API de Claude. Las pruebas de integración de las rutas usan Supertest sobre la aplicación Express real, pero sustituyen la base de datos (`jest.mock('../db')`) y la verificación del token (un doble de prueba, `test-utils/authMock.js`) para simular usuarios y roles. En las pruebas del middleware real (`auth*.test.js`) la librería `jose` también está simulada: comprueban que el servidor exige emisor y audiencia y responde 401 ante un token inválido, no que Supabase firme un token. Por eso la sección 4 repite las comprobaciones clave **contra el servidor real en producción**. El esquema de la base de datos se valida con `npm run db:probar-schema`, que sí necesita una base de datos y **no se ejecutó para este documento**.

2. Qué demuestran las pruebas respecto a la rúbrica

Cada fila cita pruebas reales, con su nombre tal como aparece en el resultado de Jest y el archivo donde viven. El generador de este documento falla si una prueba citada no existe en el resultado, de modo que no se puede citar una prueba inexistente.

Requisito	Qué se comprueba	Pruebas concretas

CRUD completo	Crear, leer, editar y borrar (noticias; igual en contenidos, productos y publicaciones del debate), con 404 si no existe.	`POST /api/noticias 201 al crear una noticia valida como admin`
		src/routes/noticias.integration.test.js · pasa
		`GET /api/noticias devuelve la lista de noticias`
		src/routes/noticias.integration.test.js · pasa
		`PUT /api/noticias/:id 200 al editar una noticia existente`
		src/routes/noticias.integration.test.js · pasa
		`DELETE /api/noticias/:id 200 al eliminar una noticia existente`
		src/routes/noticias.integration.test.js · pasa

JWT verificado en el servidor	El servidor exige emisor y audiencia del token y rechaza uno inválido con 401 (con jose simulado; ver el recuadro de la sección 1).	`verificacion del JWT con emisor y audiencia requireAuth exige iss = <proyecto>/auth/v1 y aud = authenticated`
		src/middleware/auth.jwt.test.js · pasa
		`verificacion del JWT con emisor y audiencia un token con emisor/audiencia equivocados (jose lo rechaza) responde 401`
		src/middleware/auth.jwt.test.js · pasa
		`requireAuth responde 401 si no hay encabezado Authorization`
		src/middleware/auth.test.js · pasa
		`admin con sesion aal1 (solo contrasena) pierde el poder: rol efectivo "usuario" y requireAdmin responde 403 mfa_required`
		src/middleware/auth.mfa.test.js · pasa

Rutas protegidas: 401 y 403	401 sin sesión y 403 si la sesión no tiene el rol requerido, en rutas de escritura y del panel.	`POST /api/noticias 401 sin autenticacion`
		src/routes/noticias.integration.test.js · pasa
		`POST /api/noticias 403 si el usuario no es admin`
		src/routes/noticias.integration.test.js · pasa
		`GET /api/admin/estado-seguridad 403 si el usuario no es admin ni moderador`
		src/routes/admin.integration.test.js · pasa
		`DELETE /api/foro/posts/:id (permisos) 403 si el usuario no es el autor ni admin`
		src/routes/foro.integration.test.js · pasa

Validación de datos	Los datos inválidos se rechazan con 400 antes de tocar la base de datos, incluido el bloqueo de enlaces peligrosos.	`validateNoticia rechaza titulo muy corto`
		src/validators/noticia.test.js · pasa
		`POST /api/noticias 400 si el cuerpo es invalido`
		src/routes/noticias.integration.test.js · pasa
		`validatePost rechaza esquemas peligrosos (javascript:, data:) en el enlace multimedia`
		src/validators/foroPost.test.js · pasa
		`POST /api/productos 400 si el precio es invalido`
		src/routes/productos.integration.test.js · pasa

Pagos y despliegue	El webhook de Stripe rechaza firmas falsas; la configuración de Netlify oculta los archivos internos.	`POST /api/stripe/webhook 400 si la firma es invalida`
		src/routes/stripeWebhook.test.js · pasa
		`netlify.toml el archivo interno "/build.js" se devuelve como 404`
		src/netlify-config.test.js · pasa

3. Verificación en producción desde fuera

El 24/10/2026 11:00 se ejecutaron, con `curl` desde esta computadora, 22 peticiones contra el API publicado (`sabiasquepr-backend-uufz.onrender.com`) y el sitio (`sabiasquepr.com`). Son solo lecturas públicas y peticiones **sin credenciales válidas** que el servidor debe rechazar; no crean ni modifican datos. Resultado: **22 de 22 dieron el código esperado**. La respuesta se muestra literal (recortada a 70 caracteres).

3.1 Lecturas públicas y rutas protegidas

Petición	Esperado	Obtenido	Tiempo	Respuesta literal
Lecturas públicas (deben dar 200 con datos)				
`GET /api/health`	200	**200** ✓	0,58 s	`{"ok":true,"hora_servidor":"2026-10-24T15:00:09.095Z"}`
`GET /api/noticias`	200	**200** ✓	0,38 s	`{"ok":true,"noticias":[{"id":"la-nasa-halla-el-crater-mas-grande-forma`
`GET /api/contenidos?seccion=educacion`	200	**200** ✓	0,38 s	`{"ok":true,"contenidos":[]}`
`GET /api/contenidos?seccion=revolucion-humana`	200	**200** ✓	0,34 s	`{"ok":true,"contenidos":[{"id":"recorrido-en-kart-por-una-via-costera-`
`GET /api/productos`	200	**200** ✓	0,36 s	`{"ok":true,"productos":[{"id":"bolso-puerta-san-juan","nombre":"Bolso`
Rutas protegidas SIN token (deben dar 401)				
`POST /api/noticias`	401	**401** ✓	0,29 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`PUT /api/noticias/x`	401	**401** ✓	0,30 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`DELETE /api/noticias/x`	401	**401** ✓	0,26 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`GET /api/admin/gastos`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`GET /api/admin/uso-ia`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`POST /api/foro/posts`	401	**401** ✓	0,28 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`GET /api/me`	401	**401** ✓	0,49 s	`{"ok":false,"error":"Falta token de autenticacion."}`
Rutas protegidas con un token INVENTADO (deben dar 401)				
`POST /api/noticias`	401	**401** ✓	1,35 s	`{"ok":false,"error":"Token invalido o expirado."}`
`GET /api/admin/gastos`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Token invalido o expirado."}`
`GET /api/me`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Token invalido o expirado."}`
Webhook de Stripe sin firma válida (debe dar 400)				
`POST /api/stripe/webhook`	400	**400** ✓	0,29 s	`{"ok":false,"error":"Firma de webhook invalida: No stripe-signature he`

Datos reales devueltos: 3 noticias, 11 contenidos de Revolución Humana y 7 productos. El token inventado es un JWT con firma falsa: `eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJpbnZlbnRhZG8ifQ.firma-falsa`.

Observación: Educación no tiene contenidos publicados todavía

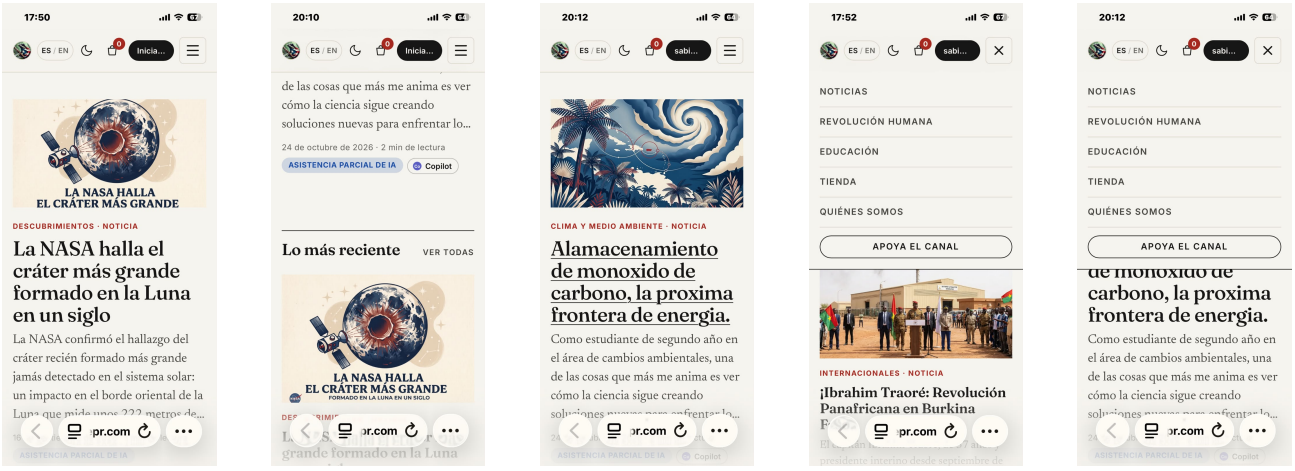
`GET /api/contenidos?seccion=educacion` responde 200 pero con la lista vacía (`{"ok":true,"contenidos":[]}`). No es un error del servidor: aún no hay piezas de Educación publicadas en la base de datos de producción, y la página lo muestra con su mensaje de «Aún no hay contenido». Conviene publicar contenido en esa sección antes de la demostración si se quiere mostrar esa ruta con datos.

4. Pruebas en navegadores y dispositivos

Qué se probó automáticamente. Se cargaron en **sabiasquepr.com** las siete páginas principales (inicio, noticias, Revolución Humana, educación, tienda, apoyo/premium y contacto) con cada navegador instalado en esta computadora, sin pantalla, controlado por el protocolo DevTools, en tres tamaños de pantalla: escritorio (1280×800), tableta (768×1024) y móvil (390×844, con emulación táctil). Por cada combinación se verificó: (1) que el documento responde 2xx, dispara «load» y muestra contenido; (2) que **no hay desbordamiento horizontal** (`scrollWidth ≤ innerWidth`); y (3) que no hay errores en la consola.

Qué cubre y qué no cubre esta prueba

Chrome y Edge (motor Chromium) y el motor de Safari (WebKit) se probaron de forma automática en Windows, con tamaños de escritorio, tableta y móvil emulados. Resultado: cargan las 7 páginas en los 3 tamaños, sin desbordamiento horizontal y sin errores de JavaScript (los avisos de seguridad que trae el distintivo «Powered by Netlify» no son del código del proyecto). **No sustituye** probar en dispositivos reales: el estudiante probó el sitio en un **iPhone 14 (Safari)**, lo que destapó dos fallos (la página se arrastraba de lado y el botón de menú quedaba cortado); se corrigieron y se volvió a verificar en el mismo iPhone (figura siguiente). Firefox, Android y Samsung Internet no se probaron.



Inicio, sin sesión · *Al bajar* · *Con sesión* · *Menú abierto* · *Menú con sesión*

Prueba manual en un iPhone 14 (Safari), 24 de octubre de 2026, después de corregir la cabecera: el botón de menú (tres líneas, o la X cuando está abierto) se ve completo, con y sin sesión iniciada, y la página ya no se arrastra de lado.

4.1 Motor de Safari (WebKit) en iPhone, iPad y Mac

Qué se probó. Con **WebKit 18.2 (Playwright)** (el motor de Safari) se cargaron en **sabiasquepr.com** las mismas siete páginas en tres tamaños con el agente de usuario de Safari: **iPhone** (390×844, táctil), **iPad** (820×1180, táctil) y **Mac** (1440×900). Por cada combinación se midió: que responda 200, que no haya errores de JavaScript, que no haya desbordamiento horizontal, que ningún campo de formulario tenga letra menor de 16 px (con menos, Safari de iPhone hace zoom solo al tocar el campo) y que no haya respuestas de red con código 400 o mayor. Además se repitió con el **almacenamiento bloqueado**, como Safari con «Bloquear todas las cookies» (tocar `localStorage` lanza `SecurityError`).

Medición (21 combinaciones: 3 dispositivos × 7 páginas)	Antes de los ajustes (2026-10-24)	Después de los ajustes (2026-10-24)
Responden 200	21 de 21	21 de 21
Sin errores de JavaScript	21 de 21	21 de 21
Sin desbordamiento horizontal	21 de 21	21 de 21
Campos de formulario de 16 px o más (sin zoom en iPhone)	12 de 21	21 de 21
Sin respuestas de red con código 400 o mayor	21 de 21	21 de 21
Con el almacenamiento bloqueado: sin errores de JavaScript	0 de 21	21 de 21

Hallazgos de esta prueba y cómo se corrigieron

1. Zoom al tocar un campo (iPhone). Los campos de texto medían 15,2 px; en 9 de 21 combinaciones había campos por debajo de 16 px (inicio, educación y contacto). Se subió la regla base a 1rem (16 px). **2. Carrito.** El cajón del carrito usaba `100vh`, que en iPhone incluye la barra de Safari y puede tapar el botón de pagar; se agregó `100dvh` con `100vh` de respaldo. **3. Almacenamiento bloqueado.** Con «Bloquear todas las cookies», 21 de 21 combinaciones lanzaban un error de JavaScript sin atrapar (el tema claro/oscuro y la confirmación de edad de las secciones 18+ leían `localStorage`/`sessionStorage` sin protección); ahora 0. Las tres correcciones se publicaron y la medición «después» se hizo contra producción. Los avisos de CSP del distintivo «Powered by Netlify» (un iframe ajeno al código del proyecto) se separan, igual que en Chrome y Edge.

Límite de esta prueba

Es el **motor** de Safari ejecutándose en una computadora con Windows, controlado de forma automática. No es un iPhone, un iPad ni un Mac reales: no cubre el teclado en pantalla, los gestos, la barra de Safari que sube y baja, ni las diferencias propias de cada versión de iOS o macOS. Por eso se hizo además la prueba manual en un iPhone 14 real.

Conclusión. 1269 de 1269 pruebas automáticas pasan (92.96 % de líneas cubiertas) y 22 de 22 comprobaciones en producción dieron el resultado esperado. Eso respalda el CRUD, las rutas protegidas, la validación de datos y la publicación en línea. Para repetirlo: `cd backend; npm test` (pruebas), `node scripts/verificar-produccion.js` (producción) y `node scripts/probar-navegadores.js` (navegadores).